GROUP 12

PROJECT SUTRA

SUPPLIER UNIFIED TENDER & REQUEST AUTOMATION

VENDOR QUOTATION MANAGEMENT SYSTEM

PROJECT SUTRA

TABLE OF CONTENTS

1 Introduction to Project SUTRA

2 Project Overview

3 Problem Statement

4 Proposed Solution

5 Users and Roles

6 Technology Stack

7 System Architecture

9 Workflow and Status Lifecycles

10 Database Design

11 Key Modules and Features

11.1 Sign-in, Vendor Registration and Approval

11.2 Master Data Management

11.3 RFQ Creation and AI Drafting

11.4 Vendor Portal and Quotation Upload

11.5 Automatic Quotation Reading

11.6 Quotation Review and Validation

11.7 Vendor Comparison and Ranking

11.8 Approval and Award Summary

11.9 Dashboard and Sahayak AI Assistant

12 AI Integration, Usage and Safety

13 Security and Data Integrity

14 Testing, Validation and Results

15 Live System Demonstration

16 Challenges Faced

17 Limitations and Future Scope

18 Conclusion

19 References

20 Q&A / Thank You

Introduction to Project SUTRA

A shared procurement thread from request to award

SUTRA

Supplier Unified Tender & Request Automation

A buyer creates an RFQ. Vendors submit PDF, Excel or Word quotations. SUTRA extracts offer details, checks them against the request, recalculates totals, compares vendors and records a manager's decision.

A "sutra" is a thread that connects the steps into one traceable process.

Project Overview

One procurement record follows the request through the final decision

01 **Purchase need**
Buyer defines items

→

02 **RFQ**
Dates and terms set

→

03 **Vendor offers**
Files submitted

→

04 **Read & validate**
Extract, match, recalculate

→

05 **Compare**
Ranked advisory report

→

06 **Approve**
Manager decides

→

07 **Audit**
History retained

CENTRAL IDEA

AI reads. The backend calculates. People decide.

Problem Statement

Manual quotation handling creates delays and inconsistent comparisons

TODAY

One request. Many formats. Manual checking.

PDF • XLS / XLSX • DOCX • Scans

01 **Typing mistakes**
Values are re-entered by hand

02 **Wrong totals**
Line items and tax are easy to miscalculate

03 **Different item names**
Equivalent products are hard to align

04 **Missing terms**
Warranty, delivery or payment details can be overlooked

05 **Scattered records**
Emails and files weaken traceability

The SUTRA Solution

Three responsibilities stay separate so the result remains reviewable

01

AI READS

Reads vendor files and extracts candidate item, price and term fields. Uncertain details become visible for review.

02

BACKEND CALCULATES

Validates fields, matches RFQ lines and recalculates line and quotation totals with decimal-safe arithmetic.

03

PEOPLE DECIDE

Buyer reviews quotations and comparison. An authorized manager records the award or rejection with a reason when required.

AI can misread a document. Money must be exact. Accountability needs a human.

Users and Roles

Access is limited by role and record ownership

ROLE	CODE	REAL-WORLD USER	RESPONSIBILITY
Administrator	ADMIN	IT / system owner	Manages accounts and master data; approves vendor sign-ups.
Buyer	PROCUREMENT	Purchasing team	Creates and manages RFQs; reviews quotations and comparisons.
Vendor	VENDOR	Supplier	Sees open RFQs and submits its own quotation and files.
Manager	APPROVER	Approving manager	Reviews the latest comparison and records award or rejection.

The gateway checks the signed user token; services also enforce role and record-level rules.

Technology Stack

Project versions are taken from the repository version matrix

01 FRONTEND

React 18.3.1 • Vite 6.4.5

React Router 6 • Axios

02 BACKEND

Java 17 • Spring Boot 4.1.2

Spring Security • Maven

03 DATABASE

PostgreSQL 18 • Flyway 12

NUMERIC money • BYTEA files

04 DOCUMENT READING

Python 3.12 • FastAPI

pdfplumber • openpyxl • python-docx • RapidFuzz

05 AI & OCR

Groq API • gpt-oss-120b

PDFBox 3 • Tesseract 5

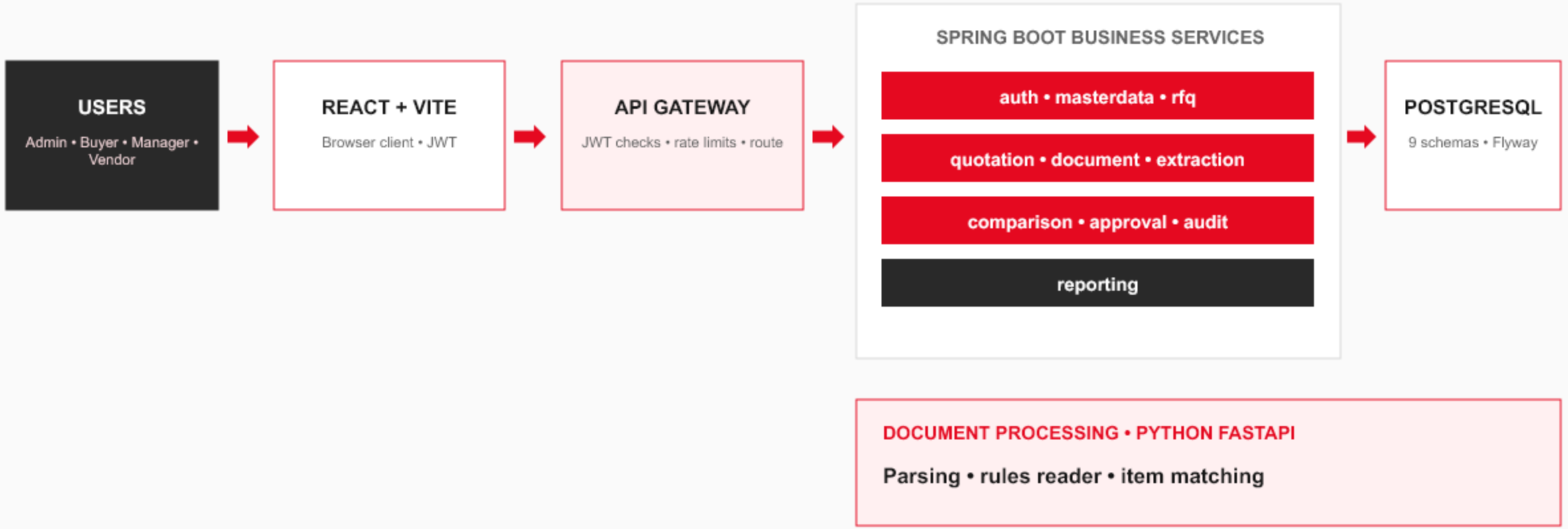
06 TESTING

JUnit 6 • Mockito 5

pytest • Vitest • React Testing Library

System Architecture

Ten Spring business services, one Spring gateway and one Python document service



Shared libraries: sutra-api + sutra-common (not deployables)

Workflow and Status Lifecycles

The RFQ and quotation state determine which actions are allowed

01 VENDOR — Sign-up → 02 ADMIN — Approve → 03 BUYER — Create RFQ → 04 VENDOR — Submit quote → 05 SYSTEM — Extract + validate → 06 BUYER — Compare → 07 MANAGER — Award / reject

RFQ STATUS

DRAFT → OPEN → UNDER REVIEW → APPROVED / REJECTED

QUOTATION STATUS

UPLOADED → PROCESSING → READY FOR COMPARISON → NEEDS REVIEW / FAILED

Only a manager's approval moves an RFQ to APPROVED or REJECTED. Warnings remain visible; only READY offers are compared.

Database Design and Integrity

PostgreSQL with service-owned schemas and cross-service relationships

9

service schemas

Flyway migrations
NUMERIC for money
BYTEA for uploaded files
Optimistic locking (@Version)
Append-only audit events
Outbox for reliable audit relay

business_entity masterdata	**purchase_request** rfq	**request_item** rfq
vendor masterdata	**quotation** quotation	**quotation_item** quotation
comparison_report comparison	**approval** approval	**app_user / role** auth

Entity → RFQ → request items

Vendor → quotation → quoted items

RFQ → comparison → approval

Key Modules and Features

Nine connected modules support the buyer, vendor and approver journey

11.1	Sign-in & vendor approval
11.2	Master data
11.3	RFQ creation & draft
11.4	Vendor portal & upload
11.5	Quotation reading
11.6	Review & validation
11.7	Comparison & ranking
11.8	Approval & award PDF
11.9	Dashboard & Sahayak

AI Integration, Usage and Safety

Java calls Groq; Python handles document parsing and rule-based helpers



AI boundaries

Reads printed values; strict schema; reports uncertainty. Prompt instructions ask it to ignore embedded document commands. These controls reduce risk but cannot guarantee prompt-injection immunity.

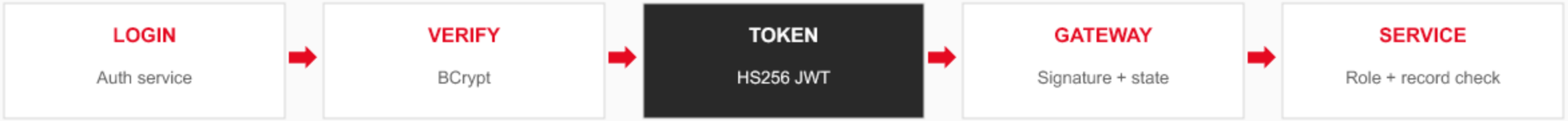
Safety and fallback

Java validates fields and recalculates every total. Rule-based extraction can be tried when Groq fails. Users can retry or enter values manually. Sahayak answers staff questions and suggests RFQ drafts; users confirm creation.

DEFAULT: openai/gpt-oss-120b **FALLBACKS: gpt-oss-20b • qwen/qwen3.8-27b**

Security and Data Integrity

Identity, file checks and audit controls protect the procurement record



AUTH	8 h user JWT • 5 min service token
ABUSE	5 failed attempts → 15 min lockout
EDGE	20 login/signup requests per min per IP
UPLOAD	sanitize + extension + signature + package check • 20 MB
INTEGRITY	**BigDecimal • NUMERIC • @Version • SHA-256 audit chain**

LIMITS: sessionStorage • no refresh token • no MFA • in-memory rate limit / lockout • no antivirus scan

Testing, Validation and Results

Recorded results shown separately; no combined total or coverage claim

FRONTEND

52 tests counted

10 files • source declarations

BACKEND

256 documented

0 failures • 0 errors • 1 skipped

PYTHON

38 passed

Parsing, rules, matching and API

ACCEPTANCE

75 / 75

Recorded end-to-end run • 10 Nov 2026

1 SKIPPED OCR TEST

PdfOcrReaderTest.readsTheTextOfAScannedPage — Tesseract was unavailable in that recorded test environment.

TOOLS JUnit • Mockito • pytest • Vitest • React Testing Library

Backend: BUILD SUCCESS (recorded) • Acceptance: prior run, not freshly executed

Frontend README says 54; current source contains 52 test declarations.

Live System Demonstration

A prepared scenario shows the buyer, vendor and manager handoffs

1

Login + dashboard

Role-specific landing

2

Buyer opens RFQ

Items and due date

3

Vendor submits offer

Prepared PDF / XLSX

4

Review extraction

Fields, totals, warnings

5

Compare vendors

Latest report and rank

6

Manager decides

Approval + history

DEMO CHECKLIST

PostgreSQL • Spring services • FastAPI • configured Groq key • Tesseract for scanned PDFs

If a dependency fails, use prepared evidence; never invent credentials or claim a live screen.

Challenges Faced

The hard parts were consistency, integration and trustworthy decisions

01	**Inconsistent documents**	Different layouts, labels and file formats.
02	**OCR uncertainty**	Scans may be low quality or missing language data.
03	**Item matching**	Vendors use different names for equivalent RFQ items.
04	**Financial validation**	Recalculate taxes and totals without trusting model arithmetic.
05	**Service integration**	Coordinate auth, document, extraction and procurement flows.
06	**Access control**	Protect role actions and vendor-specific records.
07	**Provider availability**	Handle rate limits and keep a rules-based path available.

Limitations and Future Scope

Current behavior and proposed improvements are kept separate

LIMITATIONS • CURRENT

- AI may misread unclear quotations
- Scanned PDF OCR depends on local Tesseract
- No MFA or refresh-token flow
- JWT stored in sessionStorage
- No established antivirus scanning
- Rate limits and lockouts are in-memory
- No implemented email or PO generation

FUTURE SCOPE • PROPOSED

→ MFA and stronger session management

→ Malware scanning and distributed rate limiting

→ More OCR language/layout testing

→ Email notifications and purchase orders

→ ERP integration and advanced analytics

→ Cloud deployment, monitoring and CI/CD

Conclusion and References

One system links vendor offers to a validated, reviewable decision

CONCLUSION

SUTRA brings together

- Vendor registration and RFQs
- Quotation upload and extraction
- Financial validation and comparison
- Human approval and audit history

AI READS. THE BACKEND CALCULATES. PEOPLE DECIDE.

REFERENCES

- React · react.dev/learn
- Spring Boot · docs.spring.io/spring-boot/reference
- Spring Security · docs.spring.io/spring-security/reference
- PostgreSQL · postgresql.org/docs/current
- FastAPI · fastapi.tiangolo.com
- Groq · console.groq.com/docs/api-reference
- JWT · rfc-editor.org/info/rfc7519
- JUnit · docs.junit.org/current/user-guide
- Mockito · site.mockito.org

PROJECT SUTRA

THANK YOU

QUESTIONS & ANSWERS

AI reads • The backend calculates • People decide